CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

# IT342-G2
# System Integration and Architecture

---

## System Design Document (SDD)

---

Project Title: CircuLend

Prepared By: Atillo, Ronel Cantila

Version: 0.4

Date: 3/2/2026

Status: Final

## REVISION HISTORY TABLE

| Version | Date | Author | Changes Made | Status |
|---|---|---|---|---|
| 0.1 | 2/23/2026 | Atillo, Ronel C. | Initial draft | Final |
| 0.2 | 2/28/2026 | Atillo, Ronel C. | System Architecture, and Database Design. UI/UX Designs for both Web and Mobile. API endpoints and Review | Review |
| 0.3 | 3/1/2026 | Atillo, Ronel C. | | Review |
| 0.4 | [Date] | [Your Name] | | Review |
| 0.5 | [Date] | [Your Name] | Incorporated feedback | Revised |
| 0.6 | [Date] | [Your Name] | Final review and corrections | Final |
| 1 | [Date] | [Your Name] | Baseline version for development | Approved |

TABLE OF CONTENTS

## Contents


EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

# EXECUTIVE SUMMARY

### 1.1 Project Overview

Circulend is a centralized platform to streamline the borrowing and returning of shared equipment. By integrating a Spring Boot backend, React web interface, and Android mobile application. The system utilizes JWT Authentication and Google OAuth 2.0 to secure transactions between Borrowers and Admins, ensuring real-time tracking of shared assets with integrated file handling and external data consumption.

### 1.2 Objectives

1. Develop a centralized system for tracking physical assets and their current loan status
2. Implement a cross-platform solution using a shared RESTful API for both web and mobile users.
3. Create transactional integrity for “Borrow” and “Return” actions to prevent double-booking.
4. Ensure security through an authentication layer (Bcrypt + JWT), limiting access to base on roles (Borrower/Admin).
5. Integrate the mandatory requirements, including SMTP email notifications, file uploads for item tracking, and consumption of a public external API.

### 1.3 Scope

**Included Features:**

- Authentication: User registration and authentication (email/password)
- Catalog: Real-time view of available/borrowed items
- Logic: Borrow and Return logic with status updates
- Admin: CRUD (Create, Read, Update, Delete) for items with Image uploads
- Records: Personal borrowing logs for users; global logs for Admins
- Comms: SMTP Email (Welcome & Receipts

- Platforms: Android 14 app & React Web

**Excluded Features:**

- Payments: Real payment gateway integration
- Notifications: Push notifications (mobile)
- Hardware: QR code or Barcode scanning for items
- Integrations: Social Media Login (other than Google)
- Analytics: Advanced AI/predictive analytics
- Automation: Automated inventory ordering

# 1.0 INTRODUCTION

### 1.1 Purpose

This document serves as the comprehensive design specification for the CircuLend system. It provides detailed functional and non-functional requirements, architectural decisions, API contracts, database design, and an implementation roadmap to guide development. It also outlines the integration of the mandatory features including role-based JWT authentication, Google OAuth 2.0, file uploads, external API consumption, and SMTP email notifications ensuring all components integrate seamlessly across the Spring Boot backend, the React web interface, and the native Android mobile application.

# 2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

### 2.1 Project Overview

**Project Name:** CircuLend
**Domain:** Asset Management / Resource Lending
**Primary Users:**

1. Borrowers (Students / Staff)
2. Administrators (Item Managers)

**Problem Statement:** Users need a simple, fast way to request and return shared equipment without the confusion of manual logs, while administrators need accountability and accurate status tracking.

**Solution:** A centralized application focusing on core borrowing functionality, integrated file uploads for item verification, and a consistent experience across web and mobile.

### 2.2 Core User Journeys

#### Journey 1: First-time Borrower Request

1. The user visits the CircuLend web or mobile application.
2. Click “Sign Up” or “Continue with Google” to authenticate.
3. Browse the real-time Item Catalog (viewing available items and their uploaded thumbnails).
4. Click an available item to view its specific details.
5. Click the initial “Borrow” button on the item page.
6. A Review Summary modal appears, displaying the item name, fixed pickup location, an automated borrow timestamp, and a prompt to enter the physical Asset Tag.
7. The user types the Asset Tag (e.g., DSLR-001) found on the physical item and clicks “Confirm Borrow”.
8. The system checks database availability, updates the item status to "Borrowed", creates an active transaction record, and sends an automated SMTP confirmation email.

#### Journey 2: Item Return Process

1. The user logs in with existing credentials.
2. Navigates to the “My Items” dashboard.
3. Selects the currently borrowed item and clicks “Return Item”.
4. A return modal appears prompting the user to enter the physical Asset Tag of the item they are dropping off.
5. The user enters the tag and clicks "Confirm Return".
6. The system verifies the tag matches the original transaction, records the return, updates the item back to “Available”, logs the action in the audit trail, and sends an SMTP return receipt.

#### Journey 3: Administrator Product Management

1. Admin logs in with administrative credentials.
2. Navigates to the inventory management dashboard.
3. Adds a new item, filling out details and uploading an image file, and assigning it a physical Asset Tag.
4. Views the master list of all active borrowing transactions
5. Monitors system logs and External API data (e.g., fetching current school semester dates from a public API).

### 2.3 Feature List (MoSCoW)

**MUST HAVE**

1. User authentication (Register, Login, Google OAuth, JWT, Logout)
2. Core module: Item catalog (List, Detail, Upload Image).
3. Transaction Logic (Borrow item, Return Item, Status Validation).
4. Admin panel (Item CRUD, view global transactions).
5. SMTP Email functionality (Welcome emails, transaction receipts).
6. External API integration (Displayed in the System).

**SHOULD HAVE**

1. Item categories and filtering.
2. Borrowing history dashboard for users.
3. Basic input validation feedback.
4. Responsive design for the web.

**COULD HAVE**

1. Real-time WebSocket updates when an item status changes.
2. Sandbox payment integration (for simulated late fees/deposits)
3. Order status tracking.

**WON'T HAVE**

1. Real payment processing.
2. Multi-language support.
3. Advanced analytics dashboard.

### 2.4 Detailed Feature Specifications

**Feature: User Authentication**

- **Screens:** Registration, Login, Google OAuth redirect
- **Fields:** First Name, Last Name, Email, Password, Confirm Password.
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login
- **Security:** JWT tokens, password hashing with BCrypt

**Feature: Item Catalog**

- **Screens:** Item Listing, Item Detail.
- **Display:** Grid layout, uploaded item images, names, status (Available/Borrowed).

- **API Endpoints:** `GET /api/items`, `GET /api/items/{id}`
- **Admin Functions:** `POST /api/items` (multipart/form-data for image), `PUT /api/items/{id}`, `DELETE /api/items/{id}`

**Feature: Transaction Logic (Borrow/Return)**

- **Screens:** Borrow Confirmation, My Items, Return Confirmation.
- **Process:** Validate item availability, create transaction record, update item status, trigger SMTP email.
- **API Endpoints:** `POST /api/transactions/borrow`, `POST /api/transactions/return/{id}`, `GET /api/transactions/user`

**Feature: Admin Panel**

- **Screens:** Inventory Management, Global Transactions.
- **Functions:** Add/edit/delete items with file uploads, view all user transactions.
- **Access Control:** Admin role strictly required (checked via JWT payload).

### 2.5 Acceptance Criteria

**AC-1: Successful User Registration**

- ➢ Given I am a new user
- ➢ When I enter valid email and strong password
- ➢ And confirm password matches
- ➢ And click "Create Account"
- ➢ Then my account should be created
- ➢ And I should receive an SMTP Welcome Email
- ➢ And I should be redirected to the login screen.

**AC-2: Borrowing an Item Flow**

- ➢ Given I am logged in as a Borrower
- ➢ When I select an "Available" item
- ➢ And I click "Borrow"
- ➢ And I enter the correct physical Asset Tag (e.g., DSLR-001) into the modal.
- ➢ **Then** the system should validate the tag against the database
- ➢ And the system should create a transaction record
- ➢ And the item status should immediately change to "Borrowed"
- ➢ And the UI should update to reflect the new status
- ➢ And I should receive a SMTP email receipt.
- ➢ And a borrowed confirmation should be logged in the system audit trail for the Administrator.

**AC-3: Admin Product Management**

- ➢ Given I am logged in as an Administrator
- ➢ When I add a new item with a name, description, and an image file
- ➢ And save the item
- ➢ Then the image file should be stored on the server
- ➢ And the new item should appear in the Borrower's catalog view with its image.
- ➢ And an added item confirmation should be logged in the system audit trail for the Administrator.

**AC-4: Returning an Item Flow**

- ➢ Given I am logged in as a Borrower with an "Active" borrowed item.
- ➢ When I navigate to my "My Items" dashboard and click the [Return Item] button.
- ➢ And I enter the physical Asset Tag (e.g., DSLR-001) found on the item into the return modal.
- ➢ Then the system should verify the entered tag matches the specific item assigned to that transaction.
- ➢ And the system should update the transaction status to "Completed" and the item status back to "Available".
- ➢ And the UI should immediately refresh to remove the item from my active list.
- ➢ And a return confirmation should be logged in the system audit trail for the Administrator.

# 3.0 NON-FUNCTIONAL REQUIREMENTS

## 3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests.
- Web page load time: ≤ 3 seconds on broadband.
- Database queries: complete within 500ms.

## 3.2 Security Requirements

- Authentication: JWT token passed in Authorization headers.
- Data Security: Password hashing with BCrypt.
- Authorization: API-level and UI-level role restrictions (Admin vs. Borrower).

- Vulnerability Protection: SQL injection prevention (via Spring Data JPA) and XSS protection.

### 3.3 Compatibility Requirements

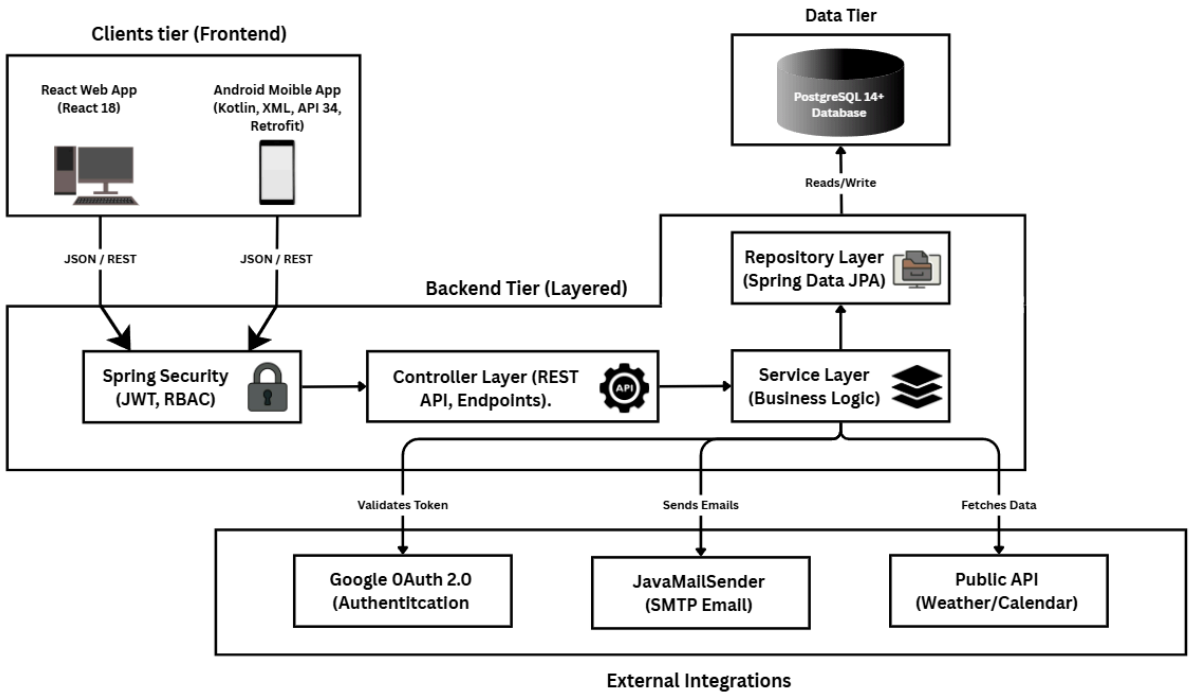
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Mobile (Android):** STRICTLY **API Level 34 (Android 14)** using Kotlin and XML-Based UI Layouts (No Jetpack Compose).
- **Networking (Mobile):** Retrofit for API integration to the Spring Boot backend.
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu

### 3.4 Usability Requirements

- Complete a borrow transaction within 3 clicks from the dashboard.
- Consistent navigation across web and mobile.
- UI must adjust based on user role.

## 4.0 SYSTEM ARCHITECTURE

### 4.1 Component Diagram




**Technology Stack:**

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, CSS, Axios
- **Mobile:** Android Kotlin, XML-Based UI Layouts (API 34), Retrofit
- **External Integrations:** Google OAuth 2.0 API, Public Weather/Calendar API, JavaMailSender (SMTP)
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Render (Backend), Vercel/Netlify (Web), APK (Mobile)

# 5.0 API CONTRACT & COMMUNICATION

## 5.1 API Standards

| | |
|---|---|
| **Base URL** | https://[server_hostname]:[port]/api/ |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer token (JWT) in Authorization header |
| **Response Structure** | {<br>"success": boolean, |

| | |
|---|---|
| | "data": object\|null,<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

**5.2 Endpoint Specifications**

# Authentication Endpoints

## User Registration

| | |
|---|---|
| **Description** | Register a new Borrower. |
| **API URL** | /auth/register |
| **HTTP Request Method** | POST |
| **Format** | JSON for all requests/responses |
| **Authentication** | None |
| **Request Payload** | {<br>"firstname": <firstname>,<br>"lastname": <lastname>,<br>"email": <email>,<br>"password": <password>,<br>} |
| **Response Structure** | {<br>"success": boolean,<br>"data": {<br>user {<br>"email": <email>,<br>"firstname": <firstname>,<br>"lastname": <lastname>,<br><br>},<br>"accessToken": <token>,<br>"refreshToken": <token>,<br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## User Login

| | |
|---|---|
| **Description** | User Login |
| **API URL** | /auth/login |
| **HTTP Method** | POST |
| **Format** | JSON for all requests/responses |
| **Authentication** | None |
| **Request Payload** | {<br>“email”: <email>,<br>“password”: <password>,<br>} |
| **Response Structure** | {<br>"success": boolean,<br>"data": {<br>“user”: {<br>“email”: <email>,<br>“firstname”: <firstname>,<br>“lastname”: <lastname>,<br>“role”: <role>,<br>},<br>“accessToken”: <token>,<br>“refreshToken”: <token>,<br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Borrow an Item

| | |
|---|---|
| **Description** | Execute Borrowing Transaction |
| **API URL** | /api/transactions/borrow |
| **HTTP Method** | POST |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | {<br>"itemId": <integer>,<br>"assetTag": "DSLR-001"<br>} |
| **Response Structure** | {<br><br> |

| | |
|---|---|
| | "success": boolean,<br><br>"data": {<br><br>"transactionId": <integer>,<br><br>"itemId": <integer>,<br>"itemName": <itemName>,<br>"borrowDate": <string>,<br>"status": <string><br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Return an Item

| | |
|---|---|
| **Description** | Return a Borrowed Item |
| **API URL** | /api/transactions/return/{id} |
| **HTTP Method** | POST |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | {<br>"assetTag": "DSLR-001"<br>} |

| | |
|---|---|
| **Response Structure** | {<br>"success": boolean,<br>"data": {<br>"transactionId": <integer>,<br>"returnDate": <string>,<br><br>"status": <string><br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Get User Transaction History

| | |
|---|---|
| **Description** | Retrieve Current User's Transaction History |
| **API URL** | /api/transactions/user |
| **HTTP Method** | GET |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | None |
| **Response Structure** | {<br>"success": boolean,<br>"data": [ |

| | |
|---|---|
| | {<br>"transactionId": <integer>,<br>"itemName": <string>,<br>"borrowDate": <string>,<br>"returnDate": <string>\|null,<br>"status": <string><br>}<br>],<br><br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Get All Items

| | |
|---|---|
| **Description** | Retrieve Item Catalog |
| **API URL** | /api/items |
| **HTTP Method** | Get |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | None |
| **Response Structure** | "success": boolean,<br>"data": [ |

| | |
|---|---|
| | {<br>"itemId": <integer>,<br>"name": <string>,<br>"category": <string>,<br>"thumbnail": <string>,<br><br>"status": <string><br>}<br>],<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Get Specific Item Details

| | |
|---|---|
| **Description** | Retrieve Specific Item Details |
| **API URL** | /api/items/{id} |
| **HTTP Method** | Get |
| **Format** | JSON for all requests/responses |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | None |
| **Response Structure** | {<br>"success": boolean,<br>"data": { |

| | |
|---|---|
| | "itemId": <integer>,<br>"name": <string>,<br>"category": <string>,<br>"description": <string>,<br>"thumbnail": <string>,<br>"status": <string><br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

## Add New Item (Admin)

| | |
|---|---|
| **Description** | Create a new item in the inventory |
| **API URL** | /api/items |
| **HTTP Method** | Post |
| **Format** | JSON / Multipart Form Data (for image upload) |
| **Authentication** | Bearer Token (JWT) |
| **Request Payload** | {<br>"name": <string>,<br>"description": <string>,<br>"categoryId": <integer>,<br>"assetTag": <string>, |

| | |
|---|---|
| | "imageFile": <file><br>}<br> |
| **Response Structure** | {<br>"success": boolean,<br>"data": {<br>"itemId": <integer>,<br>"name": <string>,<br>"assetTag": <string>,<br>"status": "AVAILABLE"<br>},<br>"error": {<br>"code": string,<br>"message": string,<br>"details": object\|null<br>},<br>"timestamp": string<br>} |

### 5.3 Error Handling

**HTTP Status Codes**

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist

- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

**Error Code Examples**

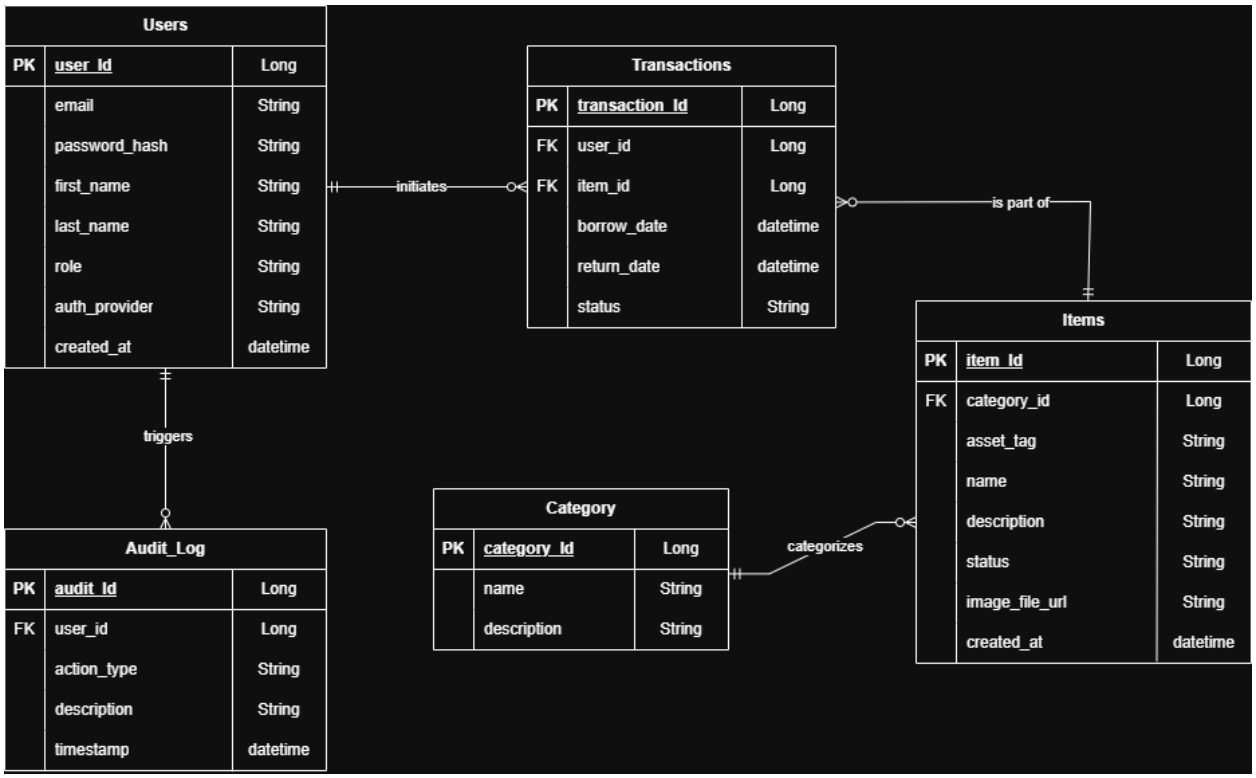
| | |
|---|---|
| Example 1 | json<br>{<br>"success": false,<br>"data": null,<br>"error": {<br>"code": "AUTH-001",<br>"message": "Invalid credentials",<br>"details": "Email or password is incorrect"<br>},<br>"timestamp": "2024-01-28T10:30:00Z"<br>} |
| Example 2 | {<br>"success": false,<br>"data": null,<br>"error": {<br>"code": "VALID-001",<br>"message": "Validation failed",<br>"details": {<br>"email": "Email is required",<br>"password": "Must be at least 8 characters"<br>}<br>},<br>"timestamp": "2024-01-28T10:30:00Z"<br>} |

**Common Error Codes**

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions (Role restriction)
- VALID-001: Validation failed
- DB-001: Item or Transaction not found
- BUSINESS-001: Item is currently unavailable/already borrowed
- BUSINESS-002: Cannot return an item you did not borrow
- SYSTEM-001: Internal server error / SMTP failure

# 6.0 DATABASE DESIGN

## 6.1 Entity Relationship Diagram




**Detailed Relationships:**

- **One-to-Many:** Category → Items (A category like "Electronics" has many items).
- **One-to-Many:** User → Transactions (A user can have multiple borrow/return records).
- **One-to-Many:** Item → Transactions (An item has a history of multiple transactions over time).

**Key Tables:**

1. **users** - Stores credentials and role (Admin/Borrower).
2. **categories** - Organizes items (e.g., Lab Equipment, Sports).
3. **items** - Core entity storing item details, current status, and file upload links.
4. **transactions** - The bridge table recording who borrowed what, when they borrowed it, and when it was returned.
5. **audit_logs** - Tracks system events (e.g., Admin added an item, physical return validations,  external API data pulls, SMTP email statuses).

**Table Structure Summary:**

- **users**: id, email, password_hash *(BCrypt)*, first_name, last_name, role, auth_provider *(LOCAL or GOOGLE)*, created_at
- **categories**: id, name, description
- **items**: id, category_id, asset_tag, name, description, status *(AVAILABLE, BORROWED, MAINTENANCE)*, image_file_url *(File Upload Requirement)*, created_at
- **transactions**: id, user_id, item_id, borrow_date, return_date *(nullable)*, status *(ACTIVE, COMPLETED)*
- **audit_logs**: id, user_id, action_type, description, timestamp

# 7.0 UI/UX DESIGN

### 7.1 Web Application Wireframes

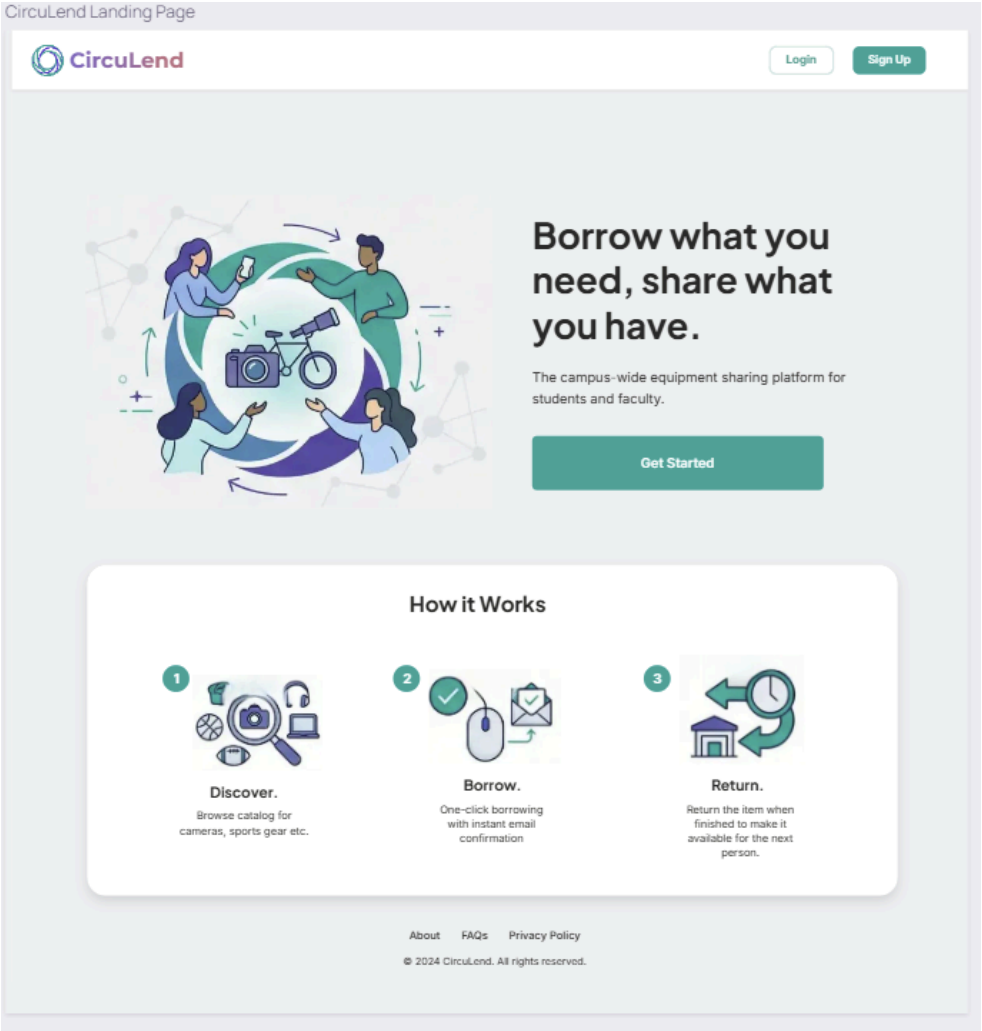
*Note: This should be wireframes from Figma*

**Landing Page**

Login

Signup

Get Started (Redirect to Signup)



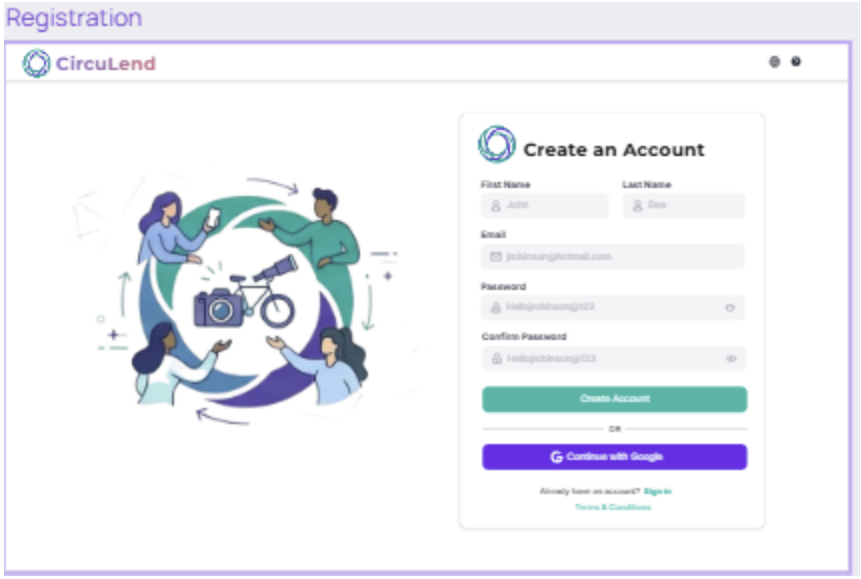

**Registration**

Create account

Continue with Google

Sign in

Terms & conditions



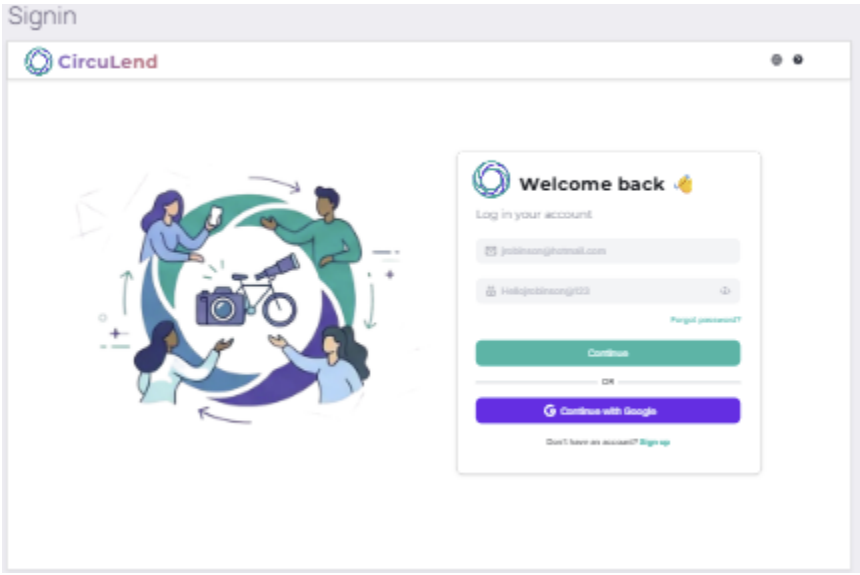

**Login**

Forgot Password

Continue (Logging in)

Continue with Google

Sign up




**Item Catalog**

Home

Dashboard

Search bar

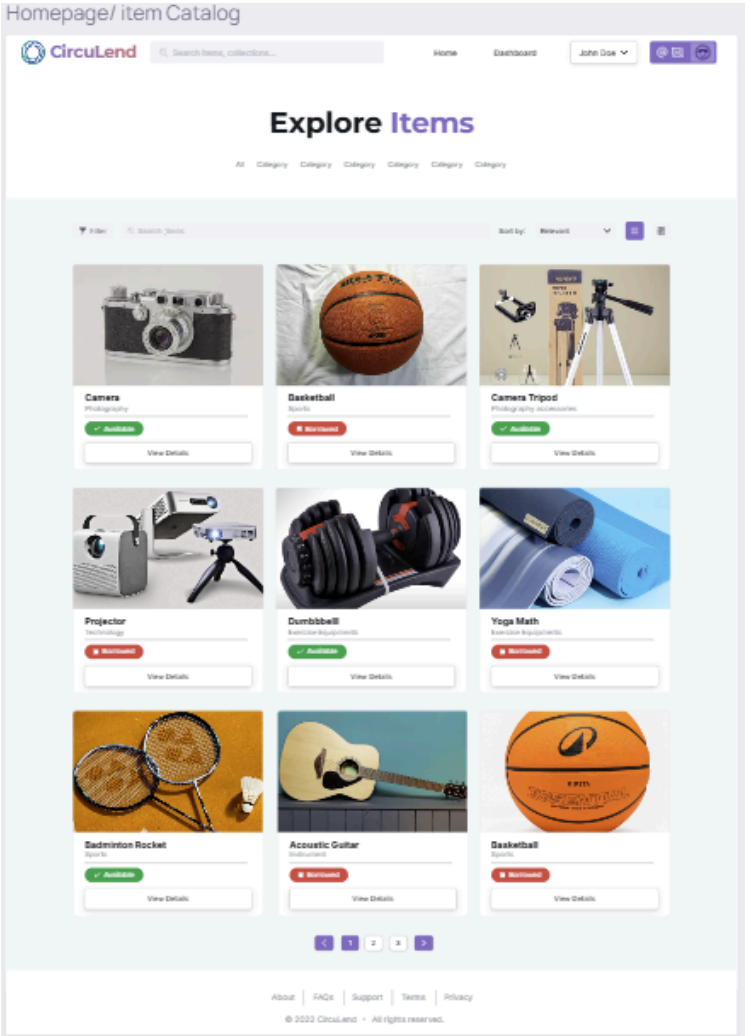
Profile Dropdown

View Details (Item Details)

Sort By (Drop down)

Next Button (To the next page of Items)

Back Button



**Item Details (Borrower)**

Home

Dashboard

Search Bar

Profile Dropdown

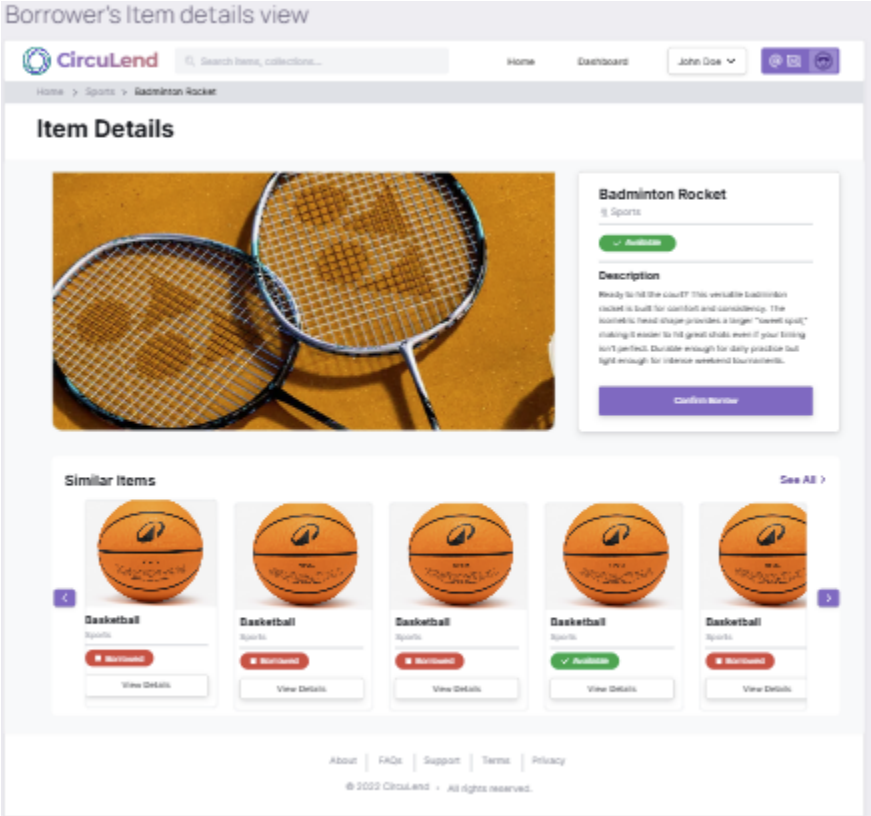
Back to Categories

Confirm Borrow

View Details

See All
Next Button for Similar Items

Previous Button for Similar Items




**Item Details (Admin)**

Home

Dashboard

Search Bar

Profile Dropdown

Back to Categories

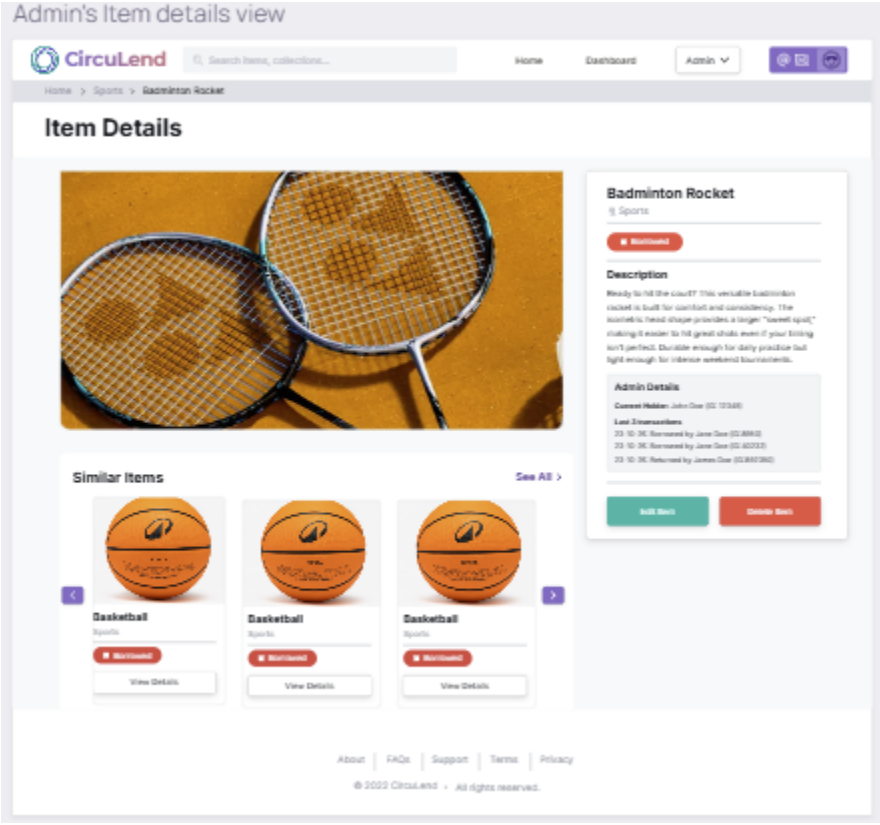
Edit Item

Delete Item

View Details

See All
Next Button for Similar Items

Previous Button for Similar Items




**Borrower Dashboard (Management/Returning)**

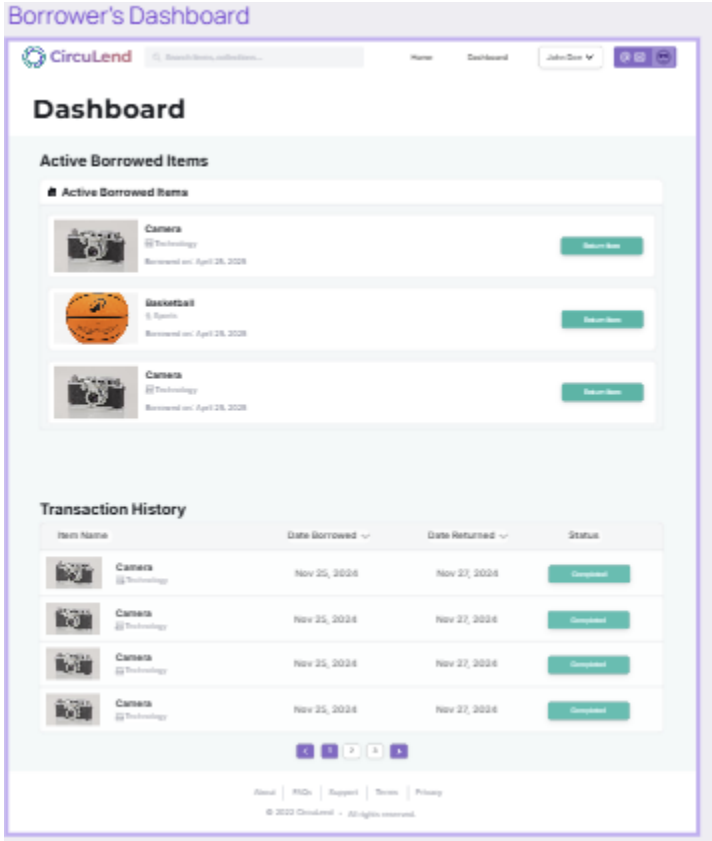
Home

Dashboard

Search Bar

Profile Dropdown

Return Item




**Admin Dashboard (Inventory/Audit Logs)**

Home

Dashboard

Search Bar

Profile Dropdown

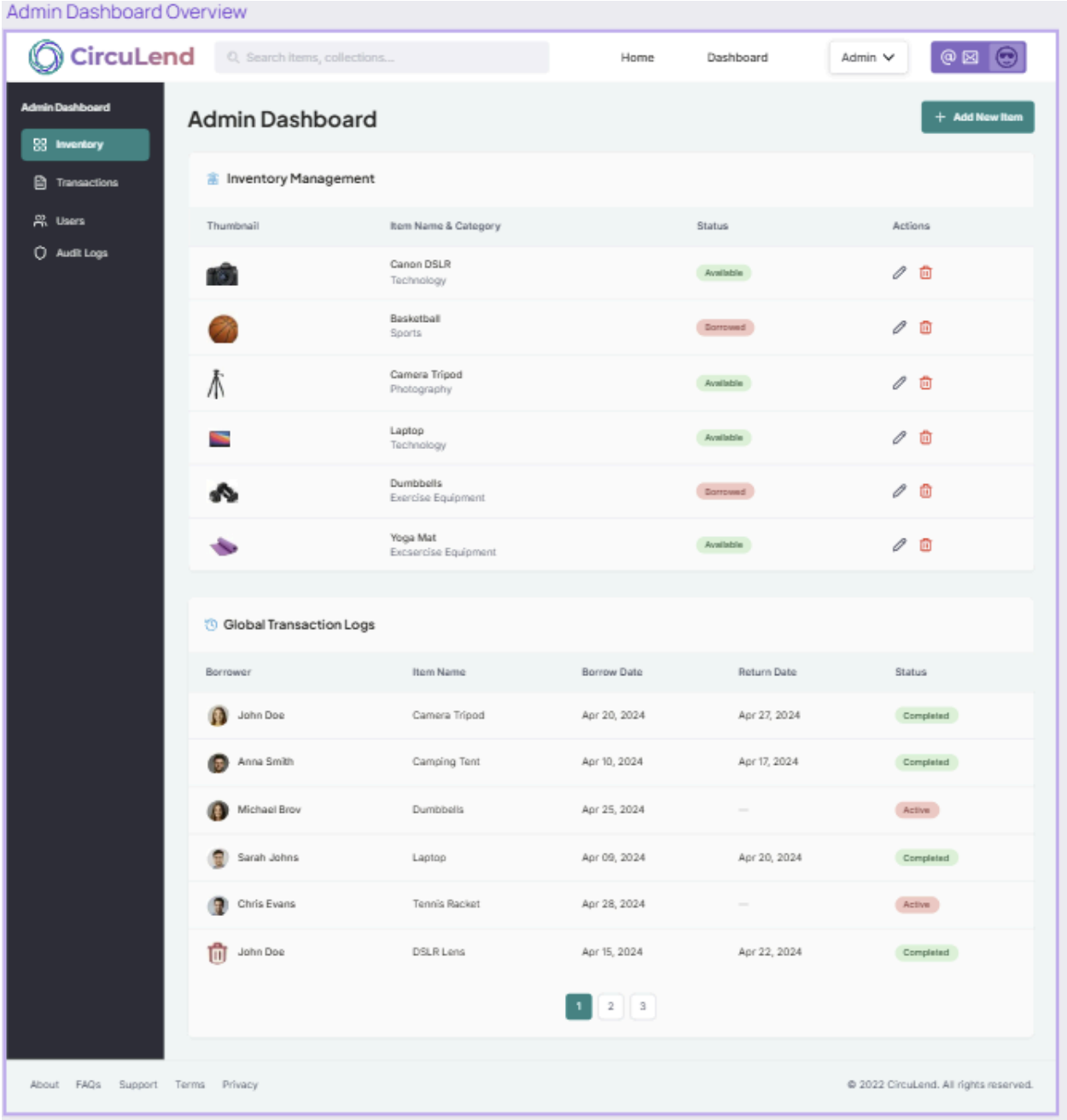
Transactions
Users
Audit Logs

Add New Item

Edit Icon

Delete Icon

(1,2,3) Pagination




## 7.2 Mobile Application Wireframes

*Note: This should be wireframes from Figma*

**Bottom Navigation**

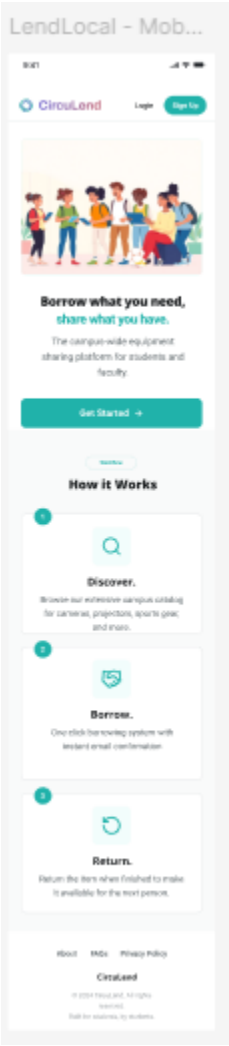
[🏠 Home] [ Dashboard] [ History] [👤 Profile]

**Landing Page**

Login

Signup

Get Started (Redirect to Signup)



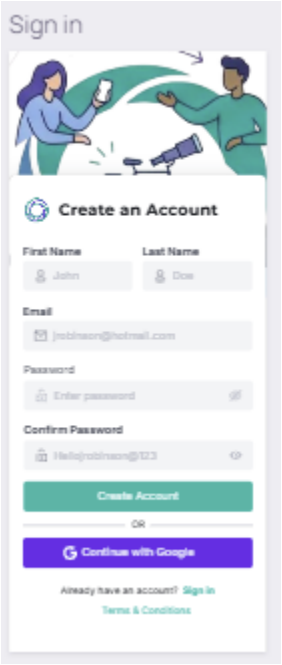

**Registration**

Create account

Continue with Google

Sign in

Terms & conditions



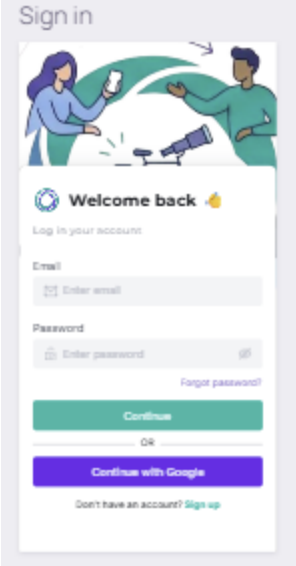

**Login**

Forgot Password

Continue (Logging in)

Continue with Google

Sign up




**Item Catalog**

Search bar

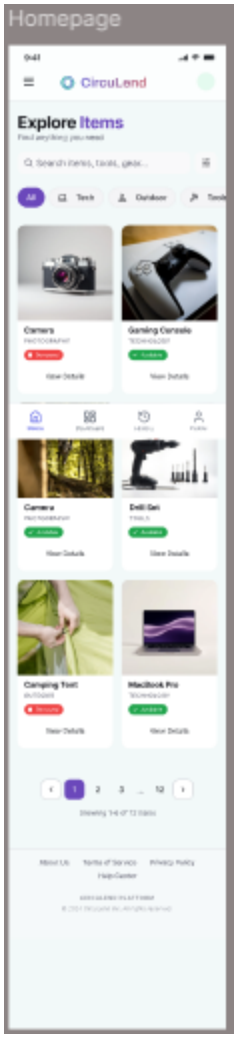
Filter

View Details (Item Details)

Next Button (To the next page of Items)

Back Button

(1,2,3,...) Pagination



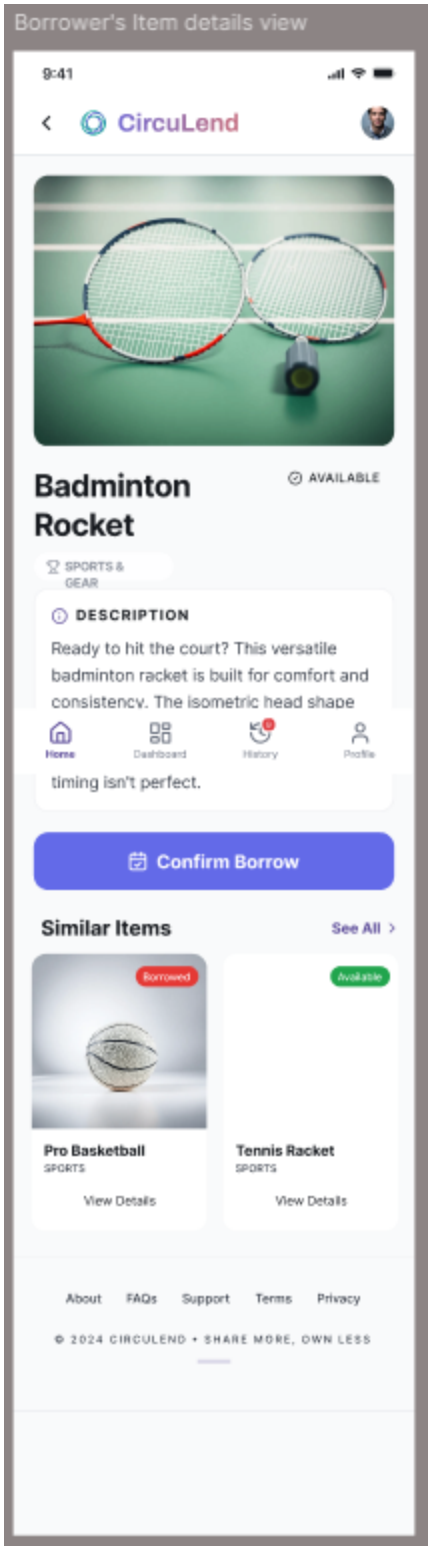

**Item Details (Borrower)**

Back Button

Confirm Borrow

View Details

See All




**Item Details (Admin)**

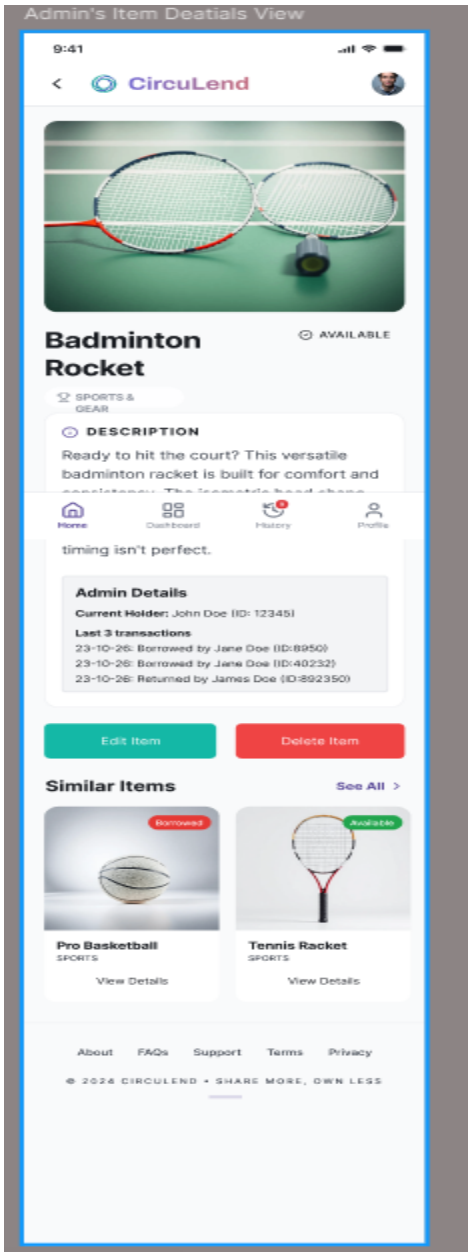
Back Button

Edit Item

Delete Item

View Details

See All



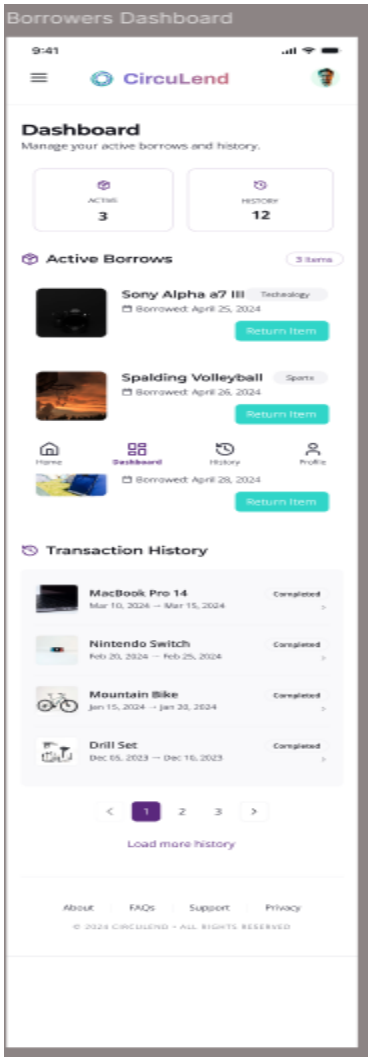
**Borrower Dashboard (Management/Returning)**

Return Item

Previous Button for Transaction History

Next Button for Transaction History

(1,2,3) Pagination




**Admin Dashboard (Inventory/Audit Logs)**

Search Bar

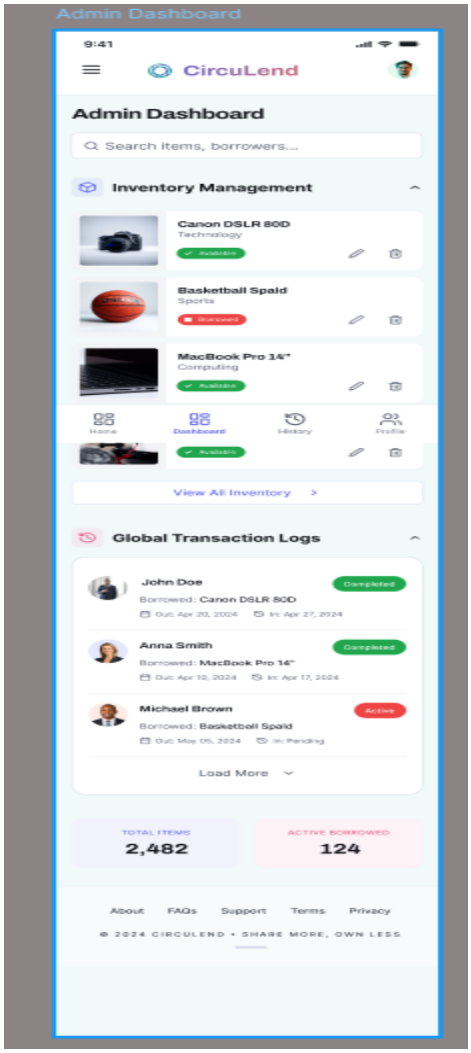
Profile Dropdown

Transactions
Users

Edit Icon

Delete Icon

(1,2,3) Pagination




**Borrow Flow**

- Step indicator: Item Details -> Review Summary (Tag Validation) -> Confirmed
- Pickup location: Fixed campus equipment room
- Borrowing summary: Bottom-sheet modal displaying item name, thumbnail, automated borrow timestamp, and a required text input field for the Asset Tag.

- Confirm borrow button: Validates the entered Asset Tag, finalizes the transaction, updates status to "Borrowed", logs the event in the audit trail, and triggers the SMTP email receipt.

**Return Flow**

- Step indicator: Dashboard -> Return Validation -> Confirmed
- Drop-off location: Fixed campus equipment room
- Return summary: Modal prompting the user to input the physical Asset Tag of the item being returned.
- Confirm return button: Verifies the tag matches the active transaction, updates the item status back to "Available", updates the transaction to "Completed", logs the event, and triggers the SMTP return receipt.

**Mobile-Specific Features:**

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

**Design System:**

- **Colors:**
  - **Primary (#14B8A6):** A professional, calming teal used for main branding, active links, and prominent action buttons (e.g., "Confirm Borrow").
  - **Secondary (#4F46E5):** An indigo used for secondary CTAs, informational elements, and alternative login options (e.g., "Google OAuth").
  - **Semantic:**
    - **Success (#10B981):** Green, used to indicate an item is "AVAILABLE" or a transaction was completed successfully.
    - **Error (#EF4444):** Red, used to indicate an item is "BORROWED" (unavailable), validation failures, or error alerts.
    - **Warning (#F59E0B):** Amber, used for warning states like "Due Soon" or pending approval.
  - **Backgrounds & Neutrals:** Gray tones (#F9FAFB, #E5E7EB, #1F2937) for backgrounds, borders, and main text.

- **Typography: Inter font family** (or similar sans-serif). Responsive sizing utilizing standard Tailwind scale (text-sm, text-base, text-lg, text-xl).
- **Spacing:** 8px grid system (e.g., p-2, p-4, p-8, space-x-4).
- **Components:** Consistent buttons (primary, secondary), input fields with validation states, standardized card layouts, and centralized modal components.
- **Responsive:** Mobile-first approach using standard breakpoints at 640px (sm), 768px (md), and 1024px (lg).

# 8.0 PLAN

**8.1 Project Timeline**

**Phase 1: Planning & Design (Completed)**

- **Week 1: Requirements & Architecture**
  - Day 1-2: Project setup, FRS, and NFR documentation.
  - Day 3-4: System architecture design (Component & Sequence Diagrams).
- **Week 2: Detailed Design**
  - Day 1-2: API specification and Database Schema (ERD).
  - Day 3-4: UI/UX Web Wireframes (Landing, Catalog, Admin).
  - Day 5-7: Mobile Wireframes and Implementation Plan finalization.

**Phase 2: Backend Development (Spring Boot)**

- **Week 3: Foundation & Security**
  - Day 1: Spring Boot initialization with PostgreSQL and SMTP dependencies.
  - Day 2: Database entities (User, Item, Category, Transaction).
  - Day 3: JWT Authentication & BCrypt implementation.
  - Day 4: User Role Management (Borrower vs. Admin).
- **Week 4: Core Logic & Transactions**
  - Day 1: Item CRUD operations with Image Upload functionality.
  - Day 2: Borrow/Return Logic (AC-2 status toggle).
  - Day 3: Search and Category filtering API.
  - Day 4: SMTP Service integration for automated receipts.
  - Day 5: API Documentation (Swagger) and Unit Testing.

**Phase 3: Web Application (React)**

- **Week 5: Frontend Foundation**
    - Day 1: React setup with Tailwind CSS for Teal/Indigo branding.
    - Day 2: Login & Registration pages with Google OAuth.
    - Day 3: Item Catalog (Explore) with real-time status badges.
    - Day 4: Item Detail Page with the 2-step Borrow Modal.
- **Week 6: Dashboards & Admin Features**
    - Day 1: Borrower Dashboard ("My Items") with Return functionality.
    - Day 2: Admin Inventory Management (Add/Edit/Delete).
    - Day 3: Global Transaction Logs & Audit Trail.
    - Day 4: Responsive UI polish for web browsers.
    - Day 5: Full API integration and End-to-End testing.

**Phase 4: Mobile Application (Android)**

- **Week 7: Android Foundation**
    - Day 1: Android Studio project structure and Retrofit setup for API calls.
    - Day 2: Mobile Auth screens (Login/Register).
    - Day 3: Mobile Catalog with Pull-to-Refresh.
    - Day 4: Mobile Borrowing Flow (Bottom-sheet modal).
- **Week 8: Mobile Polish**
    - Day 1: Mobile Dashboard ("My Items") and Return flow.
    - Day 2: Image caching implementation for offline viewing.
    - Day 3: Touch-target optimization (44x44px) and animations.
    - Day 4: Testing on physical Android devices.
    - Day 5: APK generation and mobile documentation.

**Phase 5: Integration & Deployment**

- **Week 9: Quality Assurance**
    - Day 1: Cross-platform synchronization testing (Web borrow $\rightarrow$ Mobile reflect).
    - Day 2: Security Audit (JWT expiration and Role-Based Access).
    - Day 3: Performance testing under simulated campus load.
    - Day 4: Bug fixes and UI consistency polish.
- **Week 10: Final Delivery**
    - Day 1-2: Deployment (Backend to Railway/Render; Web to Vercel/Netlify).
    - Day 3: Final SMTP mail server verification.
    - Day 4: Documentation finalization and User Manual creation.
    - Day 5: Project Submission and Defense preparation.

**Milestones:**

- **M1 (End Week 2):** SDD and Wireframes approved.
- **M2 (End Week 4):** Backend API live with functional SMTP and JWT.
- **M3 (End Week 6):** Web Application fully functional for Borrowers and Admins.
- **M4 (End Week 8):** Android APK ready for installation and testing.
- **M5 (End Week 10):** Full system deployed and integrated globally.

**Critical Path:**

1. Authentication & Role Security (Week 3): Must be solid to separate Admin and Borrower views.
2. Transaction State Machine (Week 4): Ensuring Available $\leftrightarrow$ Borrowed logic is bug-free.
3. Cross-Platform Sync (Week 9): Ensuring the database accurately reflects real-time changes across devices.

**Risk Mitigation:**

- Feature Prioritization (MVP Approach): Start with the simplest working version of the **Borrow/Return** logic before adding complex UI animations
- Early Integration Testing: Test the connection between the Spring Boot API and the PostgreSQL database in Week 3 to identify configuration issues before the Frontend development begins.
- Version Control & Backups: Maintain a consistent Git branching strategy. Always keep a stable, working version of the "Master" branch so that a functional demo is available even if a new feature causes a "Breaking Change".
- Mocking APIs for Frontend: If the Backend development is delayed, the React and Android developers will use "Mock Data" (JSON placeholders) to continue UI development without waiting for live endpoints.
- Security & Token Validation: To mitigate unauthorized access, JWT expiration and role-based route protection will be tested early in Week 9 to ensure Borrowers cannot access Admin functions.
- SMTP Fail-Safe: Since automated emails depend on external mail servers, the system will be designed to log transactions in the database *first*. If the SMTP

server is down, the transaction still completes, and the email is queued for later delivery.